

Inigo Quilez :: articles :: distance to an ellipse - 2009



Often, ellipses are usually not much more difficult than disks/circles, and one can compute lots of their properties analytically and easily, like their **bounding boxes and intersections**. However, some properties turn out to be a bit more complicated to compute than one would expect. One such property is the distance from a point to an ellipse, in 2D!

Method 1 - analytical (quartic equation)

So, for a given point in the plane $p = \{x, y\}$ and a parametric ellipse q centered in the origin with radii a and b in the x and y axes, $q(\omega) = \{a \cdot \cos \omega, b \cdot \sin \omega\}$, the square of the distance from p to the ellipse is

$$s^2(\omega) = |q(\omega) - p|^2$$

which expands to

$$s^2(\omega) = |q(\omega) - p|^2 = |q(\omega)|^2 - 2\langle q(\omega), p \rangle + |p|^2 = a^2 \cdot \cos^2 \omega + b^2 \cdot \sin^2 \omega + x^2 + y^2 - 2(a \cdot x \cdot \cos \omega + b \cdot y \cdot \sin \omega)$$

The closest point will be for an ω such that

$$\frac{ds^2(\omega)}{d\omega} = 0$$

which expands to

$$(b^2 - a^2) \cdot \sin \omega \cdot \cos \omega + ax \cdot \sin \omega - by \cdot \cos \omega = 0$$

This calls for the change of variable $\lambda = \cos(\omega)$, so that

$$\sqrt{1 - \lambda^2}((b^2 - a^2)\lambda + ax) = by\lambda$$

which expands into a quartic equation with the following coefficients:

$$\begin{aligned} k_4 &= (b^2 - a^2)^2 \\ k_3 &= 2ax(b^2 - a^2) \\ k_2 &= (a^2x^2 + b^2y^2) - (b^2 - a^2)^2 \\ k_1 &= -2ax(b^2 - a^2) \\ k_0 &= -a^2x^2 \end{aligned}$$

or by dividing the equation by the leading coefficient k_4 (the only condition to be able to do so is that the ellipse is not, in fact, a circle), and by then renaming $m = x \frac{a}{b^2 - a^2}$ $n = y \frac{b}{b^2 - a^2}$,

$$k_4 = 1$$

$$k_3 = 2m$$

$$k_2 = m^2 + n^2 - 1$$

$$k_1 = -2m$$

$$k_0 = -m^2$$

From this point on we proceed to resolve the **quartic equation** analytically with the standard method. Due to some of the symmetries in the coefficients, we get some pretty nice simplifications though. For example, the resolvent **cubic equation** loses its linear term and then depresses to

$$p^3 - 3Qp + 2R = 0$$

with

$$Q = \left(\frac{m^2 + n^2 - 1}{3} \right)^2 \quad R = 2m^2n^2 + \left(\frac{m^2 + n^2 - 1}{3} \right)^3$$

Due to all the symmetries mentioned, and the fact that we know we are looking only for real roots (possibly only one, if we are measuring the distance from a point outside the ellipse), the code gets quite compact. Also, we use the symmetry of the geometrical problem and we force our point to be on the first quadrant of the

coordinate system ($x > 0$ and $y > 0$).

```
float sdEllipse( vec2 p, vec2 ab )
{
    p = abs( p );
    if( p.x > p.y ){ p=p.yx; ab=ab.yx; }

    float l = ab.y*ab.y - ab.x*ab.x;
    float m = ab.x*p.x/l; float m2 = m*m;
    float n = ab.y*p.y/l; float n2 = n*n;
    float c = (m2+n2-1.0)/3.0; float c2 = c*c; float c3 = c2*c;
    float d = c3 + m2*n2;
    float q = d + m2*n2;
    float g = m + m *n2;

    float co;
    if( d < 0.0 )
    {
        float h = acos(q/c3)/3.0;
        float s = cos(h) + 2.0;
        float t = sin(h)*sqrt(3.0);
        float rx = sqrt( m2-c*(s+t) );
        float ry = sqrt( m2-c*(s-t) );
        co = ry + sign(l)*rx + abs(g)/(rx*ry);
    }
    else
    {
        float h = 2.0*m*n*sqrt( d );
        float s = pow(q+h, 1.0/3.0 );
        float t = c2/s;
        float rx = -(s+t) - c*4.0 + 2.0*m2;
        float ry = (s-t)*sqrt(3.0);
        float rm = sqrt( rx*rx + ry*ry );
        co = ry/sqrt(rm-rx) + 2.0*g/rm;
    }
    co = (co-m)/2.0;

    float si = sqrt( 1.0 - co*co );
    vec2 r = vec2( ab.x*co, ab.y*si );
    return length(p-r) * sign(p.y-r.y);
}
```

An interactive version (click and move the mouse over the ellipse to deform it) and reference code (click in the title of the viewport) can be found here: <https://www.shadertoy.com/view/4sS3zz>

Method 2 - Newton iterations

Unfortunately, the analytical method is both expensive and not very stable, in that the root finder is difficult to keep under control when the ellipse is close to a circle or when it is very eccentric. As a result the distance computation returns wrong values sometimes.

An alternative is to consider a numeric solver. So, let's start computing the closes point to p on the ellipse, q . We know that the vector $p-q$ will have to be perpendicular to the tangent of the ellipse at q . So, if we parametrize the ellipse again as $q(\omega) = \{ a \cdot \cos \omega, b \cdot \sin \omega \}$, then

$$\langle p - q(\omega), q'(\omega) \rangle = 0$$

with $q'(\omega)$ being the tangent, which takes this form:

$$q'(\omega) = \{ -a \cdot \sin \omega, b \cdot \cos \omega \}$$

If we expand, we get the equation

$$f(\omega) = -a \cdot \sin \omega \cdot (x - a \cdot \cos \omega) + b \cdot \cos \omega \cdot (y - b \cdot \sin \omega) = 0$$

which we are going to solve by using Newton-Raphson method, for which we need the derivative of the function above:

$$f'(\omega) = -\langle p - u, v \rangle - \langle v, v \rangle$$

with

$$u = \{ a \cdot \cos \omega, b \cdot \sin \omega \}$$

$$v = \{ -a \cdot \sin \omega, b \cdot \cos \omega \}$$

So our iterative process will be

$$\omega_{n+1} = \omega_n + \langle p - u, v \rangle / (\langle p - u, u \rangle + \langle v, v \rangle)$$

The most difficult part is ensuring that the initial value of ω , ω_0 makes the sequence converge to the desired root. An reasonable value for ω_0 can be computed by stretching the space so the ellipse becomes a circle, computing the closest point from the stretched p to the circle, and then undoing the stretch:

$$\omega_0 = \text{atan}(a \cdot y / b \cdot x)$$

Because we are going to work only with the first quadrant of the plane (since the ellipse is symmetric anyways), this will be fine. In fact, the sequence will converge for all points in the exterior of the ellipse. Unfortunately, some point in the interior will not converge, and a different initial ω_0 is required for them. In my implementation, I decided to select all the point below the line passing through the discontinuity of the gradient with slope (a, b) , and make them start at $\omega_0 = 0$.

The code below implements this method to compute the distance to an ellipse (which you can also find here:

<https://www.shadertoy.com/view/4lsXDN>:

```
float sdEllipse( vec2 p, vec2 ab )
{
    // symmetry
    p = abs( p );

    // determine in/out and initial omega value
    bool s = dot(p/ab, p/ab) > 1.0;
```

```

float w = s ? atan(p.y*ab.x, p.x*ab.y) :
             ((ab.x*(p.x-ab.x)<ab.y*(p.y-ab.y)) ? 1.5707963 : 0.0);

// find root with Newton solver
for( int i=0; i<5; i++ )
{
    vec2 cs = vec2(cos(w),sin(w));
    vec2 u = ab*vec2( cs.x,cs.y);
    vec2 v = ab*vec2(-cs.y,cs.x);
    w = w + dot(p-u,v)/(dot(p-u,u)+dot(v,v));
}

// compute final point and distance
return length(p-ab*vec2(cos(w),sin(w))) * (s?1.0:-1.0);
}

```

The code above uses 5 iterations, which produces acceptable results. For some extremely eccentric ellipses the number of iterations might need to be increased. Assuming GPUs compute sin and cos of the same angle in a single instruction, and that this instruction is very fast (which it is), then the code above is not that bad compared to the analytic solver. But it is much more robust.

A variation of the above that doesn't use trigonometric functions and uses rotations instead (which is another way to think of the same concept), is the following code (which you can also find here:

<https://www.shadertoy.com/view/tttfzr>:

```

float sdEllipse( vec2 p, vec2 ab )
{
    // symmetry
    p = abs( p );

    // initial value
    vec2 q = ab*(p-ab);
    vec2 cs = normalize( (q.x<q.y) ? vec2(0.01,1) : vec2(1,0.01) );

    // find root with Newton solver
    for( int i=0; i<5; i++ )
    {
        vec2 u = ab*vec2( cs.x,cs.y);
        vec2 v = ab*vec2(-cs.y,cs.x);
        float a = dot(p-u,v);
        float c = dot(p-u,u) + dot(v,v);
        float b = sqrt(c*c-a*a);
        cs = vec2( cs.x*b-cs.y*a, cs.y*b+cs.x*a )/c;
    }

    // compute final point and distance
    float d = length(p-ab*cs);
}

```

```
// return signed distance  
return (dot(p/ab,p/ab)>1.0) ? d : -d;  
}
```

This is the interactive version of the trigonometric solver above (click and move the mouse on the ellipse to deform the ellipse):



inigo quilez - learning computer graphics since 1994